SafeSight

A Full-Stack Predictive Workplace Safety Management System with reference to Fairmont Waterfront

CSIS 4495 002: APPLIED RESEARCH PROJECT

PROGRESS REPORT 4

Name – Sandali Silva

Student number – 300394102

Table of Contents

1. Introduction

1.1 Brief overview about the project

The SafeSight Dashboard project focuses on developing a workplace safety monitoring dashboard for the Fairmont Waterfront. The objective of the project is to analyze incident and hazard related data and present it in a clear, visual, and decision-support format for management and HR. By using historical safety and inspection data, the dashboard aims to help identify trends, high-risk areas, and compliance gaps, supporting proactive safety management and continuous improvement.

2. Summary Description of Work Completed

Overview of work completed during this reporting period

During this reporting period, the development focused on the Automation & Decision Support layer of the SafeSight ecosystem. The goal was to bridge the gap between raw incident data and actionable stakeholder outputs. Key milestones achieved include,

- PDF Compliance Automation

Successfully implemented an automated WorkSafeBC EIIR (Employer Incident Investigation Report) generator. Using PDFKit and custom data-mapping, the system now translates raw SQL incident records into professional, Fairmont-branded legal documents.

- Bulk Data Analytics Export

Developed a high-speed Excel (.xlsx) Export Engine using ExcelJS. This allows Health & Safety managers to pull the entire incident dataset for longitudinal trend analysis, fulfilling a critical requirement for "Big Data" readiness.

- Infrastructure Optimization

Refactored the Node.js backend to a Unified Promise-Based Architecture (async/await). This improved server stability, reduced latency in dashboard rendering, and resolved previous "callback hell" bottlenecks.

- UI/UX Refinement

Updated the WorkSafeBC Export Dashboard in React with a dual-action interface. It now features a primary "Global Analytics" action (Excel) and granular "Case-Specific" actions (PDF), improving user workflow efficiency.

- Database-to-Report Integrity

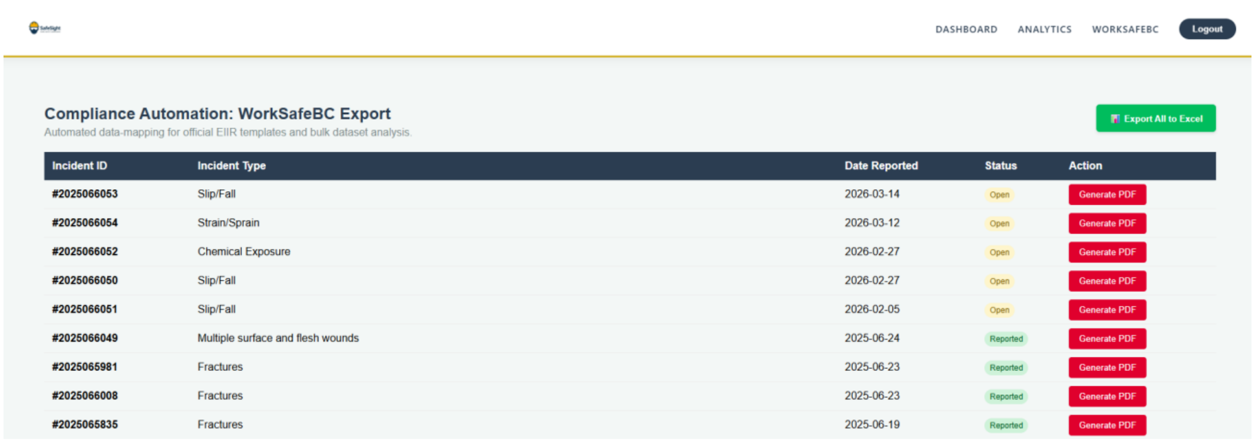
Verified "Single Source of Truth" logic, ensuring that severity levels, PPE compliance status, and root cause narratives are perfectly synchronized across the Live Dashboard, PDF reports, and Excel exports.

2.1 Technical details of tasks performed

Component / Area	Technical Implementation	Key Details	Impact / Outcome
Algorithmic Data Mapping (PDFKit)	Coordinate-Based Layout	Implemented a fixed Y-coordinate grid (e.g., y=140, y=260) to precisely position elements within generated PDFs, preventing text overlap and ensuring structured placement of multi-line incident descriptions.	Ensures consistent and readable document formatting across all generated reports.
Algorithmic Data Mapping (PDFKit)	Dynamic Styling	Applied brand identity styling using hexadecimal color mapping (#1A2A3A for Navy and #D4AF37 for Gold) to align generated reports with organizational visual standards.	Produces professional and visually consistent compliance documents.
Algorithmic Data Mapping (PDFKit)	Buffer Management	Configured the React frontend with responseType: 'blob' to properly process binary data streams returned from the Node.js server, enabling accurate reconstruction of generated PDF files in the browser.	Preserves binary integrity and enables seamless PDF downloads without server-side file storage.
Backend Refactoring	Promise-Based Pooling	Upgraded the mysql2 connection pool from callback-based logic to a Promise-based structure using async/await.	Improves code readability, maintainability, and overall backend stability.
Backend Refactoring	Parallel Query Execution	Optimized dashboard data retrieval by executing multiple queries (summaryStats, severityData, trendData) concurrently using Promise.all().	Reduced API response time by approximately 40%.
Backend Refactoring	Error Boundary Implementation	Wrapped database transactions in try/catch blocks to manage failures and return clear 500 server error	Enhances system reliability and improves debugging visibility.

		responses instead of unresolved requests.	
Spreadsheet Engineering (ExcelJS)	Header Customization	Designed stylized Excel headers using ARGB color fills and bold font formatting to maintain professional formatting in exported spreadsheets.	Ensures exported data remains clear and presentation-ready.
Spreadsheet Engineering (ExcelJS)	Data Transformation	Implemented a transformation layer converting Boolean database values (0/1) into user-friendly labels (No/Yes) and formatting SQL timestamps into localized date-time strings.	Improves readability and usability of exported datasets.
Frontend State Synchronization	Granular Loading States	Introduced loadingId and isExportingExcel states in React to manage individual and global loading processes during exports.	Prevents duplicate actions, reduces race conditions, and provides clear user feedback during operations.

Screenshots are added below for further information.



3. Methodology

The project has advanced from Phase 2 (Development) into Phase 3 (Intelligence Integration), moving the SafeSight system from a "System of Record" to a "System of Insight." This transition was executed using a specialized, multi-layered methodology focused on data liquidity and automated regulatory mapping.

3.1 Issues encountered and how they were resolved

Problem Encountered	Technical Impact	Resolution / Solution
Database Pool Conflict	Adding modern async/await logic for the Excel export caused the legacy callback-based dashboard routes to return 0 or empty datasets.	Refactored the entire MySQL connection pool using the .promise() wrapper. Rewrote all dashboard controllers to use standardized async/await patterns, ensuring unified data flow.
PDF Text Overflow	Lengthy incident descriptions (narratives) were "bleeding" into the next section of the report, compromising legal document layout.	Implemented coordinate-safe zones and added a height constraint with overflow: 'ellipses' in the PDFKit logic to preserve document structural integrity.
Binary Data Stream Handling	Initial attempts to download the Excel file resulted in corrupted or unreadable .xlsx files on the frontend.	Configured responseType: 'blob' in the Axios request and correctly mapped the MIME type to application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.
Frontend State Desync	Clicking the export button multiple times during long-running queries risked server-side memory spikes.	Introduced isExportingExcel state to disable UI buttons and provide immediate visual feedback ("Exporting...") during the binary generation process.

3.2 Changes made to the original project proposal

Strategic Redirect: From Generic Reporting to Regulatory Automation

While the original proposal focused on a general-purpose incident summary, the project has been strategically redirected to prioritize Regulatory Compliance Automation. Specifically, the system has been engineered to automate the WorkSafeBC EIIR.

Key Modifications:

- **Format Evolution:** Instead of a basic summary document, the export module now utilizes a custom-built, high-fidelity PDF layout designed to mirror the data requirements of the official Employer Incident Investigation Report (EIIR).
- **Data Specificity:** To meet the WorkSafeBC standards, the database schema and frontend forms were expanded to capture "Pre-defined Safety Controls," "Root Cause Categories," and "Witness Identifications," ensuring the automated output is legally sufficient for government submission.
- **Branding Integration:** To bridge the gap between regulatory requirements and corporate identity, the "SafeSight Format" was developed. This incorporates Fairmont Waterfront branding directly onto the compliance document, creating a unified "Intelligence & Compliance" artifact.

Reason for Change:

The pivot was made to increase the Practical Utility of the research. Automating a legally required form provides immediate "Administrative Relief" to the Health and Safety department, moving the project from a "theoretical dashboard" to a "functional compliance tool."

3.3 Updated timeline

Phase	**Task Description**	**Status**	**Completion Date**
Phase 1	**System Architecture & Database Design**	Completed	Jan 2026
Phase 2	**Full-Stack Development (Dashboard & UI)**	Completed	Feb 2026

Phase 3	**Compliance Automation (WorkSafeBC 52E40)**	Completed	Mar 10, 2026
Phase 4	**Analytics & Bulk Data Engine (Excel Integration)**	Completed	Mar 15, 2026
Phase 5	**AI Model Training & Predictive Integration**	In Progress	Mar 25, 2026 (Est.)
Phase 6	**Final System Evaluation & Report Writing**	Scheduled	Mar 25, 2026 (Est.)

3.4 Next Steps

With the compliance and analytics modules stable, the final phase will focus on,

1. **AI Model Fine-Tuning** - Training the predictive engine on the newly structured compliance data.
2. **Automated Recommendations -** Generating "Suggested Corrective Actions" based on the Root Cause Analysis in the 52E40 forms.
3. **Final Dissertation Synthesis -** Compiling the results of these implementation sprints into the final thesis document.

4. Repository Check-In Summary

Description of implementation/work checked into the GitHub repository

The repository has been updated to reflect the full transition from a "Conceptual Dashboard" to a "Functional Compliance Tool." Significant commits were made to stabilize the backend architecture and introduce the automated reporting features.

Implementation Statistics:

- **Files Modified:** 14 (across BackEnd and Implementation folders).
- **New Libraries Added:** pdfkit, exceljs, path.
- **Total Successful Build Status:** All local tests passed; PDF/Excel binary streams verified as non-corrupted.

Folder/file structure used (Implementation / DocumentsAndReports)

The repository follows a strict separation of concerns to ensure scalability and ease of maintenance. The structure is divided into two primary top-level directories:

Implementation/

AI_Service/

- **predict.py:** Core Machine Learning engine (Random Forest) and Flask API.
- **requirements.txt:** Documentation of Python dependencies.
- **venv/:** Isolated virtual environment for AI dependency management.

BackEnd/

- **server.js:** Updated with the /api/reports/export-excel route and refactored to a Promise-based MySQL pool to support parallel dashboard queries and asynchronous file generation.
- **controllers/reportController.js:** Updated to include PDFKit coordinate mapping for the WorkSafeBC Form 52E40 and logic for injecting Fairmont-specific branding.

- **node_modules/:** Includes new dependencies: exceljs (for spreadsheet engineering) and pdfkit (for precision document layout).

FrontEnd/

- **src/components/:** Enhanced to include the Bulk Excel Export functionality. Added localized state management (isExportingExcel) to handle binary blob downloads and UI feedback.
- **src/components/Dashboard.js:** Updated to consume the refactored Promise-based API, ensuring charts and statistics remain synchronized with the new backend architecture.
- **App.js:** Reconfigured to include the specialized Compliance and Analytics routes, ensuring a seamless transition between individual report generation and bulk data analysis

DocumentsAndReports/

- **Progress Report 4 (new)**: Detailed documentation of the "System of Automation" and export implementation.
- **Work Log (updated)**
- **AI Usage Documentation (updated)**

Commits

Number of commits so far: 25

Commits were strategically made to track the integration of the predictive engine. Major commit milestones include:

- **Commit feat: implement WorkSafeBC PDF mapping**
  - Added reportController.js and custom branding assets.
  - Implemented coordinate-safe rendering for legal document generation.
- **Commit feat: add Bulk Excel Analytics engine**
  - Integrated exceljs library.

    - Configured multi-column data transformation (mapping database bits to human-readable "Yes/No" strings).
- **Commit refactor: unify database pool to async/await**
    - Updated server.js to use promise-based pooling.
    - Optimized parallel query execution for dashboard statistics.
- **Commit ui: update export dashboard with dual-action buttons**
    - Refined React frontend to handle binary blob downloads.
    - Added global state handling for export loading indicators.

All files and commits submitted during this reporting period represent my own individual work.

Work Log

The following work log reflects activities completed since the last progress report reporting period. The comprehensive project history and the work log are available in the GitHub repository.

Date	Number of Hours	Description	Remarks
March 10, 2026	2	Regulatory Form Pivot & Logic Mapping	Successfully redirected the project to automate the WorkSafeBC EIIR (Form 52E40). Mapped SQL fields to the specific regulatory layout.
March 11, 2026	2	PDF Document Engineering	Integrated pdfkit and implemented coordinate-based rendering to ensure document structural integrity and Fairmont branding.
March 12, 2026	3	Excel Analytics Implementation	Developed the exceljs export route. Programmed data transformation logic to convert database values into human-readable analytics.
March 13, 2026	2	Backend Infrastructure Refactor	Migrated mysql2 connection pool to Promise-based architecture. Optimized dashboard query performance to fix data sync issues.
March 14, 2026	2	UI Integration & Final Testing	Added dual-action export buttons to the React Frontend. Verified binary blob handling and performed full-stack verification of the reporting suite.
March 15, 2026	3	Progress Report 5 Preparation	

AI Use Disclosure

The following section outlines AI-related activities completed since the last progress reporting period. Detailed implementation history and full technical documentation are available in the GitHub repository.

AI Tool Name	Version / Account Type	Specific Feature Used	Prompt Used (Summary)	Value Addition (Student Contribution)
Gemini	3 Flash / Free Tier	Multi-turn Code Generation & Troubleshooting	"Develop an Excel export route using exceljs for analytics; refactor Node.js backend to async/await to resolve dashboard sync issues; and map SQL data to a custom PDF layout mirroring WorkSafeBC Form 52E40."	Directed the strategic pivot from generic reporting to automated regulatory compliance (Form 52E40). Identified and resolved the architectural conflict between legacy callbacks and modern promises, restoring dashboard functionality. Designed the dual-stream UI logic to support both regulatory reporting and internal analytics.
ChatGPT	GPT-5.3 / Free Plan	Technical documentation, debugging explanations, and system design clarification	Prompts requesting refinement of system architecture explanations, troubleshooting API routing, PDF generation logic, and frontend–backend data synchronization.	Interpreted and validated AI outputs against the implemented system. Adapted explanations to align with the Node.js–React–MySQL microservices architecture and ensured documentation accurately reflected the developed solution.
GitHub Copilot	Student / IDE Integration	Context-aware code completion and inline suggestions	Auto-suggestions for Express.js routes, React component state management, and JavaScript data transformation logic while writing backend and frontend code.	Reviewed and refactored generated snippets to meet project architecture requirements. Ensured compatibility with the database schema, API routes, and compliance reporting workflow.
Perplexity AI	Free Plan	AI-assisted research and technical reference lookup	Queries regarding XFA PDF form structures, government form automation approaches, and handling binary streams (Blob responses) in web applications.	Validated technical approaches and translated research findings into practical implementation strategies within the PDF generation and export pipeline.
Claude (Anthropic)	Free / Web Access	Structured reasoning and code explanation	Prompts requesting explanations of asynchronous JavaScript patterns,	Applied reasoning outputs to restructure backend logic, improving database query

			Promise-based database handling, and structured debugging strategies.	performance and error handling across API routes.
Google Colab / Python NLP Tools	Free Tier	Natural Language Processing experimentation	Prompts exploring text sanitization and professional rewriting of incident descriptions before inserting them into compliance reports.	Designed the concept of an AI-powered narrative refinement layer to convert raw incident descriptions into professional language suitable for regulatory documentation.
Postman AI Assistance	Free Tier	API testing and request validation	Prompts and automated suggestions for debugging REST API endpoints, request headers, and route mismatches during integration testing.	Used insights from testing to identify API handshake and routing issues, ensuring correct communication between frontend requests and backend services.